

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour QUESTIONS: 5

Total Marks:50

Annexure A Scenario Based Assignment

Code of Conduct:

I **A . Bharath / 192512384** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A . Bharath

1. AI-Powered Drone Delivery in Flood-Affected Region i. Greedy Best-First Search (GBFS)

- Greedy Best-First Search selects the node with the lowest heuristic value (estimated distance to the goal).
- It ignores the actual travel cost and focuses only on reaching the destination quickly.

Strengths:

- Fast decision-making.
- Requires less computation.
- Suitable for urgent situations.

Weaknesses:

- May choose blocked or unsafe routes.
- Does not guarantee the shortest or safest path.
- Can produce non-optimal solutions in dynamic environments.

ii. A* Search

- A* uses the evaluation function: $f(n) = g(n) + h(n)$
 - $g(n)$: Actual cost from start.
 - $h(n)$: Estimated cost to goal.
- It considers both the path already traveled and the estimated remaining distance, producing more reliable routes.

iii. Impact of Heuristic Accuracy

- **Greedy Best-First Search:** Highly dependent on heuristic accuracy. Poor heuristics lead to poor routes.
- *A Search:** Accurate heuristics improve speed while still maintaining optimality if the heuristic is admissible.

iv. Effect of Dynamic Changes (Blocked Paths)

- Both algorithms must re-plan when paths become blocked.
- Greedy Search may repeatedly choose blocked routes.

- A* updates path costs and finds a new optimal path, though it requires more computation.
-

v. Recommended Algorithm A Search*

is the best choice because:

- Finds optimal and safer routes.
 - Handles variable movement costs.
 - Adapts better to blocked paths.
 - Provides a balance between efficiency and safety.
-

2. Smart City Traffic Signal Optimization Problem Formulation

The objective is to minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Traditional search is inefficient because:

- Huge search space.
 - Continuously changing traffic conditions.
 - High computational complexity.
-

i. Hill Climbing

Hill Climbing starts with an initial traffic signal configuration and repeatedly selects a better neighboring configuration until no improvement is possible.

Limitations

- Gets trapped in local maxima.
- Cannot guarantee the global optimum.
- Sensitive to the starting solution.

ii. Local Maxima, Plateaus, and Ridges

Local Maximum

- A traffic configuration appears best locally but is not globally optimal. **Plateau**
- Several neighboring configurations have the same performance, causing no

improvement. **Ridge**

- The best solution requires moving in multiple directions simultaneously, making progress difficult.

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing

- Occasionally accepts worse solutions.
- Escapes local maxima.
- Better exploration of the search space.

Genetic Algorithm

- Maintains multiple candidate solutions.
- Uses selection, crossover, and mutation.
- Efficiently searches large optimization problems.

iv. Recommended Approach Genetic Algorithm Reason

- Highly scalable.
- Adapts to changing traffic conditions.
- Produces high-quality solutions.
- Suitable for complex optimization problems.

3. Autonomous Mars Rover

i. Why Online Search?

The rover does not know the complete terrain before starting.

Online search:

- Makes decisions while exploring.
- Updates knowledge continuously.
- Suitable for unknown environments.

Offline search requires a complete map, which is unavailable.

ii. Challenges

Partial Observability

- Rover cannot observe the entire environment.

Dynamic Environment

- New obstacles may appear.
- Terrain conditions may change.

iii. Internal Model The rover:

-
-
- Collects sensor information.
- Stores explored locations.
- Marks hazards.
- Updates the map after every movement.
- Uses the updated information for future decisions.

iv. Comparison

Online Search	Uninformed Search	Informed Search
Learns while moving	No heuristic	Uses heuristic
Suitable for unknown maps	Explores blindly	Requires map knowledge
Highly adaptive	Slow	Faster if map exists

v. Recommended Approach

Online Search Agent

Reason

- Works without a complete map.
- Adapts to uncertainty.
- Safer navigation.
- Continuously updates knowledge.

4. University Exam Timetable (CSP) CSP Formulation Variables

- Exams (E1, E2, E3...)

-
-

Domains

Available time slots.

Available exam halls.

Constraints

- No overlapping exams for students.
- Limited halls.
- Subject order constraints.
- Faculty availability.
- Special student accommodations.

i. Variables, Domains and Constraints Each

exam is assigned:

- One time slot.
- One exam hall.
- While satisfying all constraints.

ii. Backtracking Search Backtracking:

1. Assigns a time slot.
2. Checks constraints.
3. If violated, undo assignment.
4. Try another value.
5. Continue until a valid timetable is found.

iii. Constraint Propagation (Forward Checking) Forward checking:

- Removes invalid assignments early.

- -
 - Detects conflicts before deeper search.
Reduces unnecessary backtracking.
 - Improves efficiency.
-

iv. Real-World Challenges and Best Strategy

Challenges

- Large number of courses.
- Thousands of students.
- Multiple scheduling constraints.

Best Strategy

- Backtracking with Forward Checking and Constraint Propagation.

Reason

- Efficient.
 - Scalable.
 - Produces valid timetables quickly.
-

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm Minimax:

- Maximizes AI's score.
 - Minimizes opponent's score.
 - Assumes both players play optimally.
 - Evaluates game states to select the best move.
-

ii. Computational Challenges

- Large branching factor.
Deep game tree.

-
-
- High memory usage.
- Long computation time.

Time complexity:

$O(b^d)$ where:

- **b** = branching factor
- **d** = search depth

iii. Alpha-Beta Pruning Alpha-Beta Pruning:

- Eliminates branches that cannot influence the final decision.
- Produces the same optimal result as Minimax.
- Evaluates fewer nodes.

Advantages

- Faster search.
- Lower computation.
- Allows deeper search.

Best-case complexity:

$O(b^{d/2})$

iv. Evaluation Function

The evaluation function estimates the quality of a game state using factors such as:

- Number of units remaining.
- Resource availability.
- Territory control.
- Unit health.
- Distance to objectives.
- Defensive strength.

-
-
- Attack potential.

A higher score represents a better position for the AI.

v. Recommended Strategy Alpha-Beta Pruning with Minimax Justification

- Produces near-optimal decisions.
- Reduces computation time.
- Suitable for large game trees.
- Meets real-time decision-making requirements.
- Maintains a balance between optimality and performance.